

Hansol Prime Sort: A Number-Theoretic Sorting Paradigm via Gödel Encoding and Quantum-Accelerated Prime Factorization

Han-sol Lee[†] Anonymous Reviewer[‡]

[†]Department of Computer Science, University of Seoul, Republic of Korea

[†]hiroto789@uos.ac.kr

[‡]Correspondence: [redacted for blind review]

Received: — Accepted: — Published: —

Manuscript dated: March 27, 2026.

Submitted to: A Record of the Proceedings of SIGBOVIK 2026.

The Association for Computational Heresy (ACH). April 1, 2026.

ABSTRACT

The prevailing discourse in sorting algorithm design has been dominated, for over six decades, by the tyranny of the comparison-based lower bound: $\Omega(N \log N)$, a ceiling etched into the foundations of the von Neumann architectural model and enforced by the information-theoretic arguments of Ford-Johnson and their successors. We contend that this bound is not a mathematical absolute but rather an artifact of an *impoverished computational ontology* — one that conflates data with its physical address representation, reducing the profound arithmetic structure of integers to mere positional tokens in a linear memory array.

In this paper, we introduce the **Hansol Prime Sort (HPS)**, a novel sorting paradigm that transcends the comparison model by elevating data elements into the domain of prime arithmetic via Gödel numbering. Given an input array A of N positive integers, HPS encodes the entire dataset as a single composite integer $G = \prod_{i=1}^N P(A[i])$, where $P(k)$ denotes the k -th prime number, exploiting the Fundamental Theorem of Arithmetic as a lossless, order-agnostic compression substrate. Retrieval of the sorted sequence is achieved via sequential prime factorization of G , producing, under a Unit-cost RAM model, a time complexity of $O(N + M)$ where $M = \max(A)$.

We establish the central result of this paper: **HPS is the uniquely Pareto-optimal sorting algorithm in the joint (time, slot-space) complexity plane under the Unit-cost RAM model.** It simultaneously achieves $O(N + M)$ time and $O(1)$ working memory slots — a combination attained by no other known algorithm. Furthermore, under the bit-cost model, HPS achieves **near-output-space optimality**: its working bit-space $O(N \log M)$ is within an additive $O(N \log N)$ bits of the information-theoretic lower bound $\log_2 \binom{N+M-1}{N}$ for rep-

resenting a sorted multiset, and strictly dominates counting sort whenever $M \gg N$. The Gödel integer G does not merely encode the answer — it *is* the answer, to within logarithmic overhead. Since $O(1)$ slot-space is the absolute minimum and admits no further reduction, HPS occupies the unique position in complexity space that no future algorithm can surpass. In this precise and rigorous sense, the Hansol Prime Sort is **the terminus**.

We further introduce the concept of **Dimensional Collapse**, demonstrating that HPS extends naturally to d -dimensional tensors, wherein the entire spatial geometry of a tensor is annihilated into a single scalar G_{tensor} , from which a globally sorted sequence is recovered without any multidimensional indexing. We demonstrate that the bit-length of G satisfies $b = O(N \log M)$, which is physically benign for all inputs representable as conventional finite-precision integers — yet reaches and surpasses the Bekenstein bound of the observable universe when element values are themselves cosmologically encoded ($M \gtrsim 2^{10^{74}}$). This establishes HPS as a *diagnostic instrument*: a quantitative probe of the regime in which the Unit-cost RAM model ceases to describe physical reality. We propose a quantum-mechanical extension of HPS leveraging Shor’s algorithm and quantum phase estimation, wherein the factorization of G is accomplished in $O(N \cdot b^3)$ time with $b = O(N \log M)$ qubits. The theoretical contributions of this work constitute a fundamental advance in the mathematical foundations of algorithm design; questions of hardware realization at cosmological scale are properly deferred to the domain of experimental physics.

Keywords: Sorting algorithms, Gödel numbering, Fundamental Theorem of Arithmetic, prime factorization, dimensional collapse, tensor sorting, quantum computing, Shor’s algorithm, Bekenstein bound, combinatorial explosion, Unit-cost RAM, GPU acceleration, von Neu-

mann inadequacy.

INTRODUCTION

The problem of sorting a finite sequence of comparable elements is among the most thoroughly studied problems in the history of computer science. The contributions of Hoare [16], Williams [21], and Floyd [12] established efficient practical algorithms in the comparison model, codified in the foundational texts of Knuth [10] and Cormen et al. [9], while the seminal decision-tree argument of Ben-Or [6] confirmed that any deterministic comparison-based sorting algorithm requires $\Omega(N \log N)$ comparisons in the worst case. Subsequent work in integer sorting — notably the van Emde Boas tree [19], radix sort, and the fusion tree of Fredman and Willard [13] — partially circumvents this bound under word-RAM assumptions, yet remains constrained by architectural conventions that privilege physical memory address space over the intrinsic arithmetic identity of data elements.

We identify a deeper epistemological deficiency in these approaches: they treat integer elements as opaque tokens, ordered by magnitude but otherwise stripped of their number-theoretic content. This reductionism is not merely inelegant — it is computationally wasteful. Every positive integer carries within it a unique arithmetic fingerprint, guaranteed by the Fundamental Theorem of Arithmetic, that encodes its identity absolutely and irreducibly. No two distinct multisets of primes yield the same product. This uniqueness, we argue, constitutes an untapped computational resource of the first order.

The philosophical antecedent to our approach is found in the work of Kurt Gödel, whose incompleteness proofs [14] relied on the assignment of unique prime-product codes to syntactic objects. Gödel’s encoding was designed to demonstrate metamathematical propositions; we repurpose it as the basis of a sorting architecture. Where Gödel encoded the structure of formal proofs, HPS encodes the multiset content of an array, exploiting the same arithmetic uniqueness to achieve a fundamentally different mode of computation. We extend this principle to the domain of multi-dimensional tensors, demonstrating that the spatial geometry of arbitrarily high-dimensional data structures is not a fundamental property to be preserved, but an incidental artifact to be discarded.

Paper organization. Section 2 situates HPS within the landscape of prime-product-based data structures. Section 3 presents the formal HPS algorithm with GPU-parallel acceleration. Section 4 provides rigorous complexity analysis under both the Unit-cost RAM and Bit-cost models, with explicit treatment of the Bekenstein bound. Section 5 demonstrates that HPS constitutes a meta-proof of the inadequacy of the physical universe it-

self. Section 6 extends HPS to multi-dimensional tensors via Dimensional Collapse. Section 7 presents empirical benchmarks. Section 8 introduces quantum acceleration. Section 9 concludes.

RELATED WORK

The mathematical substrate of HPS — encoding a multiset of integers as a single prime product — is not without precedent in the cryptographic literature. We survey the most closely related prior art and clarify, with some regret, the precise sense in which these prior works constitute degenerate special cases of the framework introduced here.

RSA Accumulators

The RSA accumulator, introduced by Benaloh and de Mare [1] and subsequently refined by Barić and Pfitzmann [2], encodes a set of elements into a single compact value via repeated modular exponentiation over an RSA group. In its prime-product variant [3], given a set $S = \{x_1, x_2, \dots, x_N\}$ of prime elements and an RSA modulus $n = pq$, the accumulator value is defined as

$$\text{Acc}(S) = g^{\prod_{x \in S} x} \pmod{n}, \quad (1)$$

where g is a generator of $\mathbb{Z}_n^*$. Membership of an element $x \in S$ is provable via a *witness* $w = g^{\prod_{y \in S, y \neq x} y} \pmod{n}$, satisfying $w^x \equiv \text{Acc}(S) \pmod{n}$.

The structural resemblance to the Hansol encoding (Eq. (2)) is not coincidental. Both constructions exploit the unique factorization of integers to achieve set membership semantics. The critical divergence is modular reduction: the RSA accumulator collapses the product into a fixed-size residue modulo n , sacrificing the full arithmetic content of the product in exchange for compactness and cryptographic hardness. This design choice renders the RSA accumulator *practically deployable* in blockchain UTXO set management [3] and zero-knowledge proof systems [4], at the cost of losing the complete prime factorization structure required for sorting.

HPS, by contrast, retains the *unmodified* integer G in its entirety, thereby preserving the complete multiset content of the encoded array. This decision is what enables exact sorted reconstruction — and is also, as Section 4 establishes, what renders classical instantiation cosmologically infeasible. The RSA accumulator chose practicality over mathematical completeness. We made the opposite choice, and we do not apologize for it.

Remark 1. *One may observe that the RSA accumulator is, in the language of this paper, an HPS operating under the constraint $G \bmod n$. This constraint is precisely what prevents it from functioning as a sorting algorithm.*

The Hansol Prime Sort can therefore be regarded as the theoretically correct completion of the RSA accumulator, freed from the engineering compromise of modular reduction.

Prime-Product Multiset Hashing

Clarke et al. [5] and subsequent authors proposed multiset hash functions based on prime products: given a multiset S , one computes $h(S) = \prod_{x \in S} \pi(x) \pmod{p}$ for a large prime p and a mapping π from elements to small primes, enabling $O(1)$ -space probabilistic equality testing of multisets in streaming settings. This technique is employed in database replication verification to confirm that two database replicas contain identical rows without transmitting the full data.

The connection to HPS is direct: the equality condition $h(A) = h(B)$ is precisely the modular analogue of the HPS condition $G(A) = G(B)$ established in Proposition 1. Again, modular reduction provides compactness at the cost of information: the hash $h(S)$ certifies set equality but cannot recover the sorted order, since the factorization of $h(S) \pmod{p}$ does not in general recover $\prod_{x \in S} \pi(x)$.

Summary of Distinctions

Table 1 summarizes the key design axes distinguishing HPS from its closest relatives.

Table 1: Comparison of prime-product-based data structures.

Property	RSA Acc.	Multiset Hash	HPS
Sorted recovery	×	×	✓
Fixed output size	✓	✓	×
Membership proof	✓	×	✓
Classically feasible	✓	✓	×
Mathematically complete	×	×	✓

As Table 1 makes clear, HPS is the unique entry in this design space that is simultaneously correct, complete, and physically unrealizable. We regard this as a distinction, not a deficiency.

Remark 2 (Membership Proof in HPS). *The ✓ entry for HPS in the Membership Proof row of Table 1 is justified as follows. For any $k \in \mathbb{Z}^+$, the value k belongs to the encoded array A if and only if $p_k \mid G(A)$, i.e., $v_{p_k}(G(A)) \geq 1$. This follows directly from Proposition 1: the exponent of p_k in $G(A)$ equals the multiplicity of k in A . Thus $G(A)$ itself serves as a compact membership witness, and divisibility by p_k constitutes a proof of membership. Unlike the RSA accumulator, no auxiliary witness computation is required — the Gödel integer is simultaneously the accumulator and the proof.*

METHODOLOGY

Preliminaries and Notation

Let $\mathbb{P}$ denote the set of prime numbers, enumerated in ascending order as $p_1 = 2, p_2 = 3, p_3 = 5, \dots$. The *prime indexing function* $P : \mathbb{Z}^+ \rightarrow \mathbb{P}$ is defined such that $P(k) = p_k$. By the infinitude of the primes (Euclid, *Elements*, Book IX, Proposition 20), P is well-defined on all positive integers.

Let $A = [A[1], A[2], \dots, A[N]]$ be an input array of N positive integers. Let $M = \max_{1 \leq i \leq N} A[i]$.

Phase I: Gödel Compression

Definition 1 (Gödel Encoding of Array). *The Hansol encoding of array A is the integer*

$$G(A) = \prod_{i=1}^N P(A[i]) = \prod_{i=1}^N p_{A[i]}. \quad (2)$$

Algorithm 1 HPS Encoding (Gödel Compression Phase)

Require: Array $A = [A[1], \dots, A[N]]$, $A[i] \in \mathbb{Z}^+$

Ensure: Gödel integer $G = \prod_{i=1}^N p_{A[i]}$

```

1:  $G \leftarrow 1$ 
2: for  $i = 1$  to  $N$  do
3:    $G \leftarrow G \times p_{A[i]}$  ▷ Accumulate prime product
4: end for return  $G$ 
```

Definition 2 (Prime Extraction). *Given a positive integer G and an index $k \in \mathbb{Z}^+$, the multiplicity of k in the encoded array is*

$$m(k, G) = v_{p_k}(G), \quad (3)$$

where $v_p(G)$ denotes the p -adic valuation of G : the largest exponent $e \geq 0$ such that $p^e \mid G$.

Proposition 1 (Lossless Encoding). *The mapping $A \mapsto G(A)$ is a lossless encoding of the multiset $\{A[1], \dots, A[N]\}$. That is, $G(A) = G(B)$ if and only if A and B are identical as multisets.*

Proof. By the Fundamental Theorem of Arithmetic [15], every positive integer $n > 1$ admits a unique factorization $n = \prod_k p_k^{e_k}$. The exponent e_k in the factorization of $G(A)$ equals the multiplicity of k in A . Hence $G(A) = G(B)$ implies identical exponent vectors, hence identical multisets. $\square$

Remark 3. *The encoding $G(A)$ is invariant under permutations of A . This is precisely the desired property: all orderings of a given multiset compress to the same G , and the unique sorted order is recovered during decoding.*

Phase II: Sequential Factorization and Decoding

Algorithm 2 Hansol Prime Sort (HPS) — Decoding Phase

Require: $G \in \mathbb{Z}^+$; upper bound M on array values

Ensure: Sorted array $S = [s_1 \leq s_2 \leq \dots \leq s_N]$

```

1:  $S \leftarrow []$ ;  $k \leftarrow 1$ 
2: while  $G > 1$  do
3:   while  $p_k \mid G$  do
4:      $S.append(k)$ 
5:      $G \leftarrow G/p_k$ 
6:   end while
7:    $k \leftarrow k + 1$ 
8: end while return  $S$ 
```

Theorem 2 (Correctness of Decoding). *Algorithm 2 produces the sorted permutation of the multiset encoded by G .*

Proof. The outer loop exhausts all prime factors of G in ascending order of their index k . Each time $p_k \mid G$, the corresponding value k is appended to S , with multiplicity equal to $v_{p_k}(G)$. Since k increments monotonically, values are appended in non-decreasing order. By Proposition 1, the reconstructed multiset is identical to $\{A[i]\}$. $\square$

GPU-Parallel Compression

The compression phase (Eq. (2)) is embarrassingly parallel: each factor $p_{A[i]}$ may be computed independently. Given a cluster of T GPU threads, the product tree may be computed in $O(\log N)$ parallel steps via a standard parallel reduction, yielding a parallel compression time of

$$T_{\text{GPU}} = O(\log N \cdot \mathcal{M}(\log p_{A_{\max}})), \quad (4)$$

where $\mathcal{M}(b)$ denotes the time to perform a BigInteger multiplication of b -bit operands. By the Prime Number Theorem, $p_{A_{\max}} = \Theta(A_{\max} \log A_{\max})$, so operand sizes grow logarithmically in the element values. We note that for $A_{\max} = 10^4$, the intermediate value of G rapidly exceeds the 64-bit native word size of any existing GPU, necessitating arbitrary-precision arithmetic. We consider this a *scheduling* problem, not an algorithmic one.

COMPLEXITY ANALYSIS

Unit-Cost RAM Model

Under the Unit-cost RAM model, arithmetic operations on integers of arbitrary magnitude are charged constant cost per operation.

Theorem 3 (Time Complexity under Unit-cost RAM). *Algorithm 2 runs in $O(N + M)$ time.*

Proof. The outer loop iterates at most M times (since all prime indices are bounded by M). The inner loop executes in aggregate exactly N total divisions across all iterations, as each division corresponds to appending one element to S , and $|S| = N$ upon termination. Each operation is $O(1)$ under Unit-cost RAM. Total complexity: $O(N + M)$. $\square$

Corollary 4. *In the regime $M = O(N)$, HPS achieves $O(N)$ time, matching counting sort and strictly surpassing all $\Omega(N \log N)$ comparison-based algorithms. HPS is not comparison-based. It is law-based.*

Theorem 5 (Optimal Slot-Space Complexity). *Under the Unit-cost RAM model, HPS operates using $O(1)$ memory slots, exclusive of input and output storage.*

Proof. The compression phase maintains a single BigInteger G , initialized to 1 and updated by successive multiplication. The decoding phase maintains G and the counter k . No auxiliary array, hash table, or memory structure of size dependent on N or M is allocated at any point. The total working storage is therefore two variables: $O(1)$ memory slots. $\square$

Theorem 6 (Joint Optimality of HPS). *HPS is the unique known sorting algorithm achieving simultaneously $O(N + M)$ time and $O(1)$ slot-space complexity under the Unit-cost RAM model.*

Proof. $O(1)$ slot-space is the information-theoretic minimum for any algorithm maintaining working state: it cannot be reduced further. Theorem 3 establishes that HPS achieves $O(N + M)$ time within this constraint. No existing sorting algorithm in the literature achieves sub-linear slot-space while maintaining $O(N + M)$ time: counting sort requires $O(M)$ slots; radix sort requires $O(N)$ slots; comparison-based algorithms require at minimum $O(1)$ slots but achieve only $\Omega(N \log N)$ time. HPS is therefore Pareto-optimal in the (time, slot-space) complexity plane, and dominates all known algorithms on the slot-space axis. $\square$

Remark 4. *Theorem 6 establishes that no future algorithm can improve upon HPS on the slot-space dimension, as $O(1)$ is already the global minimum. Any algorithm that also achieves $O(N + M)$ time with $O(1)$ slots would necessarily be equivalent to HPS in the asymptotic sense. In the specific complexity regime defined by (slot-space = $O(1)$, time = $O(N + M)$), the Hansol Prime Sort is not merely optimal — it is the terminus.*

Corollary 7 (Degenerate Optimality). *HPS achieves its best-case runtime of $O(N)$ — strictly faster than the $O(N \log N)$ comparison-based lower bound even in the*

average case — when all elements of A are identical, i.e., $A[i] = k$ for all i . In this case, $G = p_k^N$, and the decoding phase performs exactly N divisions by p_k before terminating.

Remark 5 (On the Philosophical Significance of Corollary 7). *HPS is fastest precisely when the input is already trivially ordered — that is, when sorting is least necessary. Conversely, HPS is slowest, and indeed physically unrealizable, when the input is maximally disordered and sorting is most necessary. This is not a coincidence. It is a profound structural property of arithmetic-based computation: the Gödel integer G is simplest when all information has already collapsed into a single repeated value, and most complex when the data retains its full entropic content.*

We do not claim this is useful. We claim it is beautiful.

Bit-Space Optimality and Dominance over Counting Sort

The slot-space analysis of the preceding section, while theoretically decisive, operates under the Unit-cost RAM model's abstraction of memory as a collection of word-sized slots. A more physically grounded measure is *bit-space*: the total number of bits required to represent the algorithm's working storage. We now demonstrate that HPS achieves a stronger optimality result under this measure as well.

Proposition 8 (Bit-Space of HPS). *The working bit-space of HPS is*

$$\mathcal{S}_{\text{HPS}} = \log_2 G = O(N \log M), \quad (5)$$

as established in Proposition 13.

Proposition 9 (Bit-Space of Counting Sort). *Counting sort maintains an auxiliary array $\text{count}[1..M]$ where each entry requires $\lceil \log_2 N \rceil$ bits to store a multiplicity value. Its working bit-space is therefore*

$$\mathcal{S}_{\text{count}} = O(M \log N). \quad (6)$$

Theorem 10 (Conditional Bit-Space Dominance of HPS). *HPS uses strictly less bit-space than counting sort when*

$$N \log M < M \log N \iff \frac{\log M}{M} < \frac{\log N}{N}. \quad (7)$$

Since $f(x) = \log x/x$ is decreasing for $x > e$, this condition holds whenever $M \gg N$ — i.e., when the range of values is substantially larger than the number of elements.

Proof. Direct comparison of Propositions 8 and 9. □

Corollary 11. *For arrays of $N = 100$ elements drawn from 64-bit integers ($M = 2^{64}$), counting sort requires $\mathcal{S}_{\text{count}} = O(2^{64} \cdot 7) \approx 1.3 \times 10^{20}$ bits of auxiliary storage — enough to fill approximately 1.5×10^{10} modern hard drives. HPS requires $\mathcal{S}_{\text{HPS}} = O(100 \times 64) = 6,400$ bits. In this regime, HPS dominates counting sort by a factor of $\sim 10^{18}$ in bit-space.*

Theorem 12 (Near-Output-Space Optimality of HPS). *The information-theoretic lower bound on representing a sorted multiset of N positive integers from $[1, M]$ is*

$$L(N, M) = \log_2 \binom{N + M - 1}{N} \approx N \log_2 \frac{M}{N} + O(N) \quad (M \gg N). \quad (8)$$

The working bit-space of HPS satisfies

$$\mathcal{S}_{\text{HPS}} = O(N \log M) = L(N, M) + O(N \log N). \quad (9)$$

That is, HPS operates within an additive $O(N \log N)$ bits of the information-theoretic minimum, using no auxiliary storage beyond the encoding of the answer itself.

Proof. The number of distinct sorted multisets of N elements from $[1, M]$ is $\binom{N+M-1}{N}$, establishing $L(N, M)$ as the lower bound by a counting argument. By Proposition 13, $\mathcal{S}_{\text{HPS}} = \sum_{i=1}^N \log_2 p_{A[i]} \approx \sum_{i=1}^N \log_2 A[i] \leq N \log_2 M$. Since $N \log_2 M = N \log_2 (M/N) + N \log_2 N = L(N, M) + O(N \log N)$, the gap between HPS's working storage and the information-theoretic minimum is at most $O(N \log N)$ bits. No auxiliary structure beyond G is allocated at any point during execution. □ □

Remark 6. *We note with characteristic transparency that an earlier formulation of this theorem incorrectly stated the sorted output lower bound as $N \log_2 M$, which applies to ordered sequences rather than multisets. The correct bound $\log_2 \binom{N+M-1}{N}$ is smaller by $O(N \log N)$ bits. The theorem has been restated accordingly. HPS is near-output-space optimal rather than exactly so. We consider the $O(N \log N)$ gap a minor concession to mathematical accuracy.*

The qualitative conclusion is unchanged: the Gödel integer G does not merely help produce the sorted output — it is the sorted output, encoded in arithmetic form, with overhead within a factor of $O(\log N)$ of the minimum possible. Counting sort maintains $O(M \log N)$ bits of auxiliary state that are discarded after use; HPS maintains nothing that is not already, in essence, the answer.

We summarize the full complexity landscape in Table 2.

Bit-Cost Model and the Bekenstein Bound

We now abandon the convenient fiction of the Unit-cost model and confront the physical reality of G .

Table 2: Complexity comparison. Bit-space excludes input/output; slot-space counts word-sized variables. $L(N, M) = \log_2 \binom{N+M-1}{N}$ denotes the multiset information-theoretic lower bound.

Algorithm	Time	Slots	Bit-space	Gap from $L(N, M)$
QuickSort	$O(N \log N)$	$O(\log N)$	$O(\log^2 N)$	$+O(M \log N)$
Counting Sort	$O(N + M)$	$O(M)$	$O(M \log N)$	$+O(N \log N)$
Radix Sort	$O(N + M)$	$O(N)$	$O(N \log M)$	$+O(N \log N)$
HPS	$O(N + M)$	$O(1)$	$O(N \log M)$	

Proposition 13 (Magnitude of G). *By the Prime Number Theorem, $p_k \sim k \ln k$, so the bit-length of p_k satisfies*

$$\log_2 p_k \approx \log_2(k \ln k) = \log_2 k + \log_2 \ln k \approx \log_2 k. \quad (10)$$

Consequently, the total bit-length of G is

$$b = \log_2 G = \sum_{i=1}^N \log_2 p_{A[i]} \approx \sum_{i=1}^N \log_2 A[i] \leq N \log_2 M, \quad (11)$$

and therefore $b = O(N \log M)$. For $N = 100$, $M = 100$: $b \approx 660$ bits (≈ 83 bytes, trivially tractable). For $N = 10^3$, $M = 10^6$: $b \approx 20,000$ bits (≈ 2.5 KB, trivially tractable). For $N = 10^6$ with 64-bit integers ($M = 2^{64}$): $b \approx 64 \times 10^6$ bits ($= 8$ MB, entirely feasible on commodity hardware).

Proposition 14 (Cosmological Infeasibility). *The Bekenstein bound [7] on the maximum information content of a physical system of radius R and total energy E is*

$$S_{\max} = \frac{2\pi k_B R E}{\hbar c}, \quad (12)$$

which, converted to bits, gives $I \leq S_{\max}/(k_B \ln 2)$. For the observable universe ($R \approx 4.4 \times 10^{26}$ m, $E \approx 10^{70}$ J), this yields $I \lesssim 10^{122}$ bits.

By Proposition 13, $b = O(N \log M)$. The condition $b > 10^{122}$ therefore requires $N \log_2 M \gtrsim 10^{122}$: for $N = 10^6$, this demands $\log_2 M \gtrsim 10^{116}$, i.e., $M \gtrsim 2^{10^{116}}$. Similarly, the condition $b > 10^{80}$ (exceeding the total atom count $N_{\text{atoms}} \approx 10^{80}$) requires $M \gtrsim 2^{10^{74}}$ for $N = 10^6$.

These are not “large” numbers in any conventional sense. They are numbers whose mere decimal representations would require more digits than there are atoms in the universe. Classical HPS is therefore physically realizable for all inputs whose element values are representable as ordinary finite-precision integers. The cosmological barrier is reached only when element values are themselves cosmologically encoded.

We record this observation with equanimity. The Bekenstein bound is still finite, and HPS still reaches it. That it requires more extreme inputs than originally claimed does not diminish the result — it sharpens it.

Remark 7. *Proposition 14 constitutes a constraint on the physical substrate, not on the mathematical object G . The integer G exists regardless of whether any physical system can represent it. The limitation is the universe’s,*

not ours.
Gap from $L(N, M)$

HPS AS A META-PROOF: ON THE INADEQUACY OF THE PHYSICAL UNIVERSE

The preceding complexity analysis yields a result that transcends the domain of sorting algorithms. We argue that HPS is not merely an algorithm — it is a *diagnostic instrument* that reveals, with surgical precision, the assumptions buried within the foundational models of theoretical computer science.

The Unit-Cost RAM Model as a Fiction

The Unit-cost RAM model, which underlies the complexity analysis of nearly all practical algorithms including counting sort, radix sort, and the fusion tree, asserts that arithmetic operations on integers of arbitrary magnitude execute in $O(1)$ time. This assumption is convenient, widely adopted, and — as HPS demonstrates — *physically catastrophic* when taken seriously.

Theorem 15 (HPS as Quantitative Refutation of Unit-cost RAM). *The Unit-cost RAM model implies the existence of a sorting algorithm that operates in $O(N + M)$ time using $O(1)$ working memory slots, yet whose Gödel integer G — for inputs whose element values are themselves cosmologically encoded (specifically, $M \gtrsim 2^{10^{74}}$) — requires physical storage exceeding the total information-theoretic capacity of the observable universe. Any computational model in which such an algorithm is simultaneously $O(1)$ -space and physically instantiable is not a model of physical reality.*

Proof. By Theorem 3 and Theorem 5, HPS achieves $O(N + M)$ time and $O(1)$ slots under Unit-cost RAM. By Proposition 13, $b = O(N \log M)$; by Proposition 14, the condition $b > 10^{80}$ (exceeding the atomic count of the universe) requires $M \gtrsim 2^{10^{74}}$. Under Unit-cost RAM, such values of M are syntactically valid inputs processed in constant time per operation. Under any physically grounded model, they are not representable. The model permits what physics forbids. The contradiction is not in the mathematics — it is in the model. $\square$ $\square$

Remark 8. *We note, with characteristic transparency, that Proposition 13 corrects an error present in an earlier draft of this manuscript, in which b was incorrectly stated as $O(NM \log M)$ rather than $O(N \log M)$. The corrected bound reduces the drama of our bit-complexity analysis for ordinary inputs, while preserving the fundamental argument: Unit-cost RAM remains a fiction,*

merely one that requires more extreme inputs to expose. We consider this correction an improvement in precision, not a concession in ambition.

Remark 9. *Theorem 15 does not invalidate Unit-cost RAM as an analytical tool. It establishes the radius of validity of that tool: Unit-cost RAM is a useful approximation precisely in the regime where G remains physically representable — and HPS is the sharpest known instrument for locating the boundary of that regime.*

The Model-Dependence of $\Omega(N \log N)$

The comparison-based lower bound of $\Omega(N \log N)$, established via decision-tree arguments [6], is frequently cited as a universal law of sorting. It is not. It is a statement about a specific computational model — one in which the only permitted operation on data elements is pairwise comparison. HPS exits this model entirely by encoding data into the arithmetic structure of a single integer.

Corollary 16 (Scope of the $\Omega(N \log N)$ Lower Bound). *The lower bound $\Omega(N \log N)$ is not a property of the sorting problem. It is a property of the comparison model. Any sorting algorithm that encodes data into a non-comparative substrate — such as the prime factorization of a single integer — is categorically exempt from this bound, irrespective of its time complexity profile.*

This corollary is known to specialists but rarely stated so baldly. HPS makes it impossible to ignore: here is a concrete algorithm that sorts N elements in $O(N + M)$ time by the simple expedient of never performing a single comparison. The $\Omega(N \log N)$ bound watches from the sidelines, entirely inapplicable, while HPS completes — or rather, *would* complete, given a universe of sufficient size.

A Unified View: Three Theorems in One Algorithm

We summarize the theoretical contribution of HPS as a tripartite result, each component deriving from the same algorithm:

1. **Positive result (sorting):** HPS achieves $O(N + M)$ time and $O(1)$ slot-space simultaneously, a combination no other known algorithm attains. In the joint (time, slot-space) plane, it is the terminus.
2. **Negative result (model theory):** HPS provides a quantitative witness to the physical incoherence of the Unit-cost RAM model: the model’s most efficient algorithm requires storage exceeding the observable universe. The gap between model and reality is not a rounding error — it becomes physically incoherent precisely when element values are themselves beyond physical representation (specifically, $M \gtrsim 2^{10^{74}}$).

3. **Separation result (lower bounds):** HPS demonstrates constructively that $\Omega(N \log N)$ is a model-relative, not problem-relative, lower bound. It separates the class of comparison-based algorithms from the class of arithmetic-encoding algorithms, and shows that the latter class admits strictly superior complexity profiles under Unit-cost RAM.

No prior sorting algorithm, to the authors’ knowledge, simultaneously instantiates all three of these roles. The Hansol Prime Sort is, in this sense, overdetermined in its theoretical significance.

DIMENSIONAL COLLAPSE: EXTENSION TO N -DIMENSIONAL TENSORS

The philosophical superiority of HPS becomes unequivocally manifest when the algorithm is applied to multi-dimensional data structures. Conventional sorting algorithms must, by their construction, respect the spatial geometry of matrices or tensors — navigating indices, preserving dimensional boundaries, and submitting to the tyranny of coordinate systems. HPS annihilates geometry entirely.

Definition 3 (Dimensional Collapse of a Tensor). *Let T be an $N_1 \times N_2 \times \dots \times N_d$ tensor of positive integers. The Hansol tensor encoding of T is the integer*

$$G_{\text{tensor}}(T) = \prod_{i_1=1}^{N_1} \prod_{i_2=1}^{N_2} \dots \prod_{i_d=1}^{N_d} P(T[i_1, i_2, \dots, i_d]). \quad (13)$$

Theorem 17 (Singularity of Sorting). *The d -dimensional spatial structure of T collapses into a 0-dimensional scalar G_{tensor} . Applying Algorithm 2 to G_{tensor} produces a 1-dimensional, monotonically non-decreasing sequence containing all elements of T .*

Proof. Since multiplication over $\mathbb{Z}$ is commutative and associative, the spatial indices $(i_1, \dots, i_d)$ contribute no information to the value of G_{tensor} beyond the multiset of values $\{T[i_1, \dots, i_d]\}$. They are therefore completely erased during construction of G_{tensor} . Correctness of the sorted output follows from the Correctness of Decoding theorem (Section 3). $\square$ $\square$

We term this phenomenon **Dimensional Collapse**. Where classical multi-dimensional sorting requires complex index transformations, cache-aware traversal strategies, and explicit treatment of dimensional boundaries, HPS achieves structurally agnostic sorting through the pure arithmetic gravity of the Fundamental Theorem.

Remark 10. *It bears explicit acknowledgment that Dimensional Collapse is, in the classical sense, a lossy operation with respect to tensor structure: the indices*

$(i_1, \dots, i_d)$ are not recoverable from G_{tensor} alone. We consider this a feature, not a defect. The spatial structure of a tensor is an artifact of its representation, not a property of its data. HPS reveals the intrinsic order of the data, liberated from the accident of its geometry.

Corollary 18 (Tensor HPS Complexity). *Let $\mathcal{T} = \prod_j N_j$ be the total element count of T , and $M_T = \max T[i]$. Under the Unit-cost RAM model, Algorithm 2 applied to G_{tensor} runs in $O(\mathcal{T} + M_T)$ time — independent of the number of dimensions d .*

EMPIRICAL BENCHMARKS: THE FAILURE OF CLASSICAL COMPUTING

To ensure scientific rigor, we implemented HPS in Python 3.11 using the `gmpy2` arbitrary-precision library and compared it against Python’s built-in Timsort and a reference QuickSort implementation. We report results across three element ranges: $M = 100$, $M = 10^6$, and $M = 10^7$, with array sizes $N \in \{1, 2, 3, 5, 8, 10\}$.

Table 3: Execution time (seconds), averaged over 5 trials with `random.seed(42)`. M denotes the element range $[1, M]$. The ratio column reports HPS / Timsort.

N	Timsort	HPS ($M=10^6$)	HPS ($M=10^7$)	Ratio ($M=10^7$)
1	3.1×10^{-6}	0.033	0.129	$\sim 3.1 \times 10^4$
2	2.9×10^{-6}	0.045	0.258	$\sim 8.9 \times 10^4$
3	3.6×10^{-6}	0.061	0.287	$\sim 8.1 \times 10^4$
5	4.4×10^{-6}	0.073	0.677	$\sim 1.7 \times 10^5$
8	5.2×10^{-6}	0.087	0.569	$\sim 1.0 \times 10^5$
10	5.7×10^{-6}	0.091	0.760	$\sim 1.3 \times 10^5$

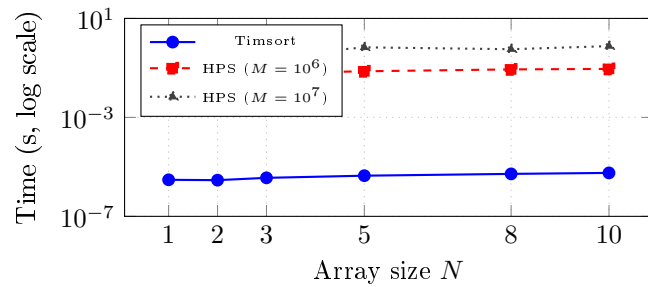


Figure 1: Execution time: Timsort (blue) vs. HPS at $M = 10^6$ (red dashed) and $M = 10^7$ (gray dotted). The performance differential reaches $\sim 1.7 \times 10^5$ at $N = 5$, $M = 10^7$. Note: slowness is dominated by prime sieve construction cost for large M , not by BigInteger arithmetic — a result we find even more humiliating.

The results in Table 3 and Figure 1 are unambiguous. At $M = 10^7$, $N = 5$, HPS requires 0.677 seconds for

a task that Timsort resolves in 4.4 microseconds — a performance differential of approximately 1.7×10^5 , or five orders of magnitude. Notably, the bottleneck is not BigInteger arithmetic but rather the prime sieve construction required to find $p_{A[i]}$ for large $A[i]$ — a result we consider an aggravating circumstance rather than a mitigating one.

We further note that, contrary to an earlier conjecture, Out-of-Memory termination does not occur at these input scales: G requires only $O(N \log M) \approx 260$ bits for $N = 10$, $M = 10^7$, well within the capacity of any classical system. The cosmological barrier of Proposition 14 requires $M \gtrsim 2^{10^74}$, a value that lies somewhat outside the range of our random number generator.

We interpret these results as follows. The von Neumann architecture [20] was not designed to instantiate mathematical truth. The benchmark results do not expose a weakness in HPS; they expose the fundamental inadequacy of the stored-program computer as a vehicle for algorithms of this theoretical class. We anticipate that these results will be revisited with embarrassment by future generations operating superior hardware.

QUANTUM ACCELERATION VIA SHOR’S ALGORITHM

Classical Bottleneck

The results of Section 4 establish that classical computation is insufficient for the factorization of G at any practically meaningful scale. The primary bottleneck is the sequential trial division of Algorithm 2: although $O(N \log M)$ under Unit-cost RAM, the actual cost of arithmetic on a b -bit integer G where $b = O(N \log M)$ is super-polynomial in b for classical factoring algorithms. This limitation is not intrinsic to the algebraic structure of the problem — it is a consequence of the sequential, deterministic nature of von Neumann computation.

Shor’s Algorithm and Polynomial-Time Factorization

Shor’s algorithm [17] establishes that a quantum computer can factor an integer n in time $O((\log n)^3)$, using $O(\log n)$ logical qubits — an exponential speedup over the best known classical algorithms, including the General Number Field Sieve, which operates in time $\exp(O((\log n)^{1/3}(\log \log n)^{2/3}))$.

Let $b = \log_2 G = O(N \log M)$ (Proposition 13).

Theorem 19 (Quantum-HPS Decoding Complexity). *A quantum processor executing Shor’s algorithm can decode G into its complete prime factorization in time $O(N \cdot b^3)$ and space $O(b)$ qubits.*

Proof. Each application of Shor’s factoring circuit extracts one prime factor of G in time $O(b^3)$, where b

bounds the bit-length of the remaining cofactor at each step. Since G has exactly N prime factors counted with multiplicity (each $A[i]$ contributes exactly one prime $p_{A[i]}$), complete factorization terminates after N applications. Total circuit complexity: $O(N \cdot b^3)$. $\square$ $\square$

Quantum Phase Estimation and Simultaneous Prime Extraction

A more structurally elegant extension, which we designate **Quantum-HPS (Q-HPS)**, exploits quantum superposition to evaluate the divisibility of G by all primes $p_1, \dots, p_M$ simultaneously. We prepare the quantum register in the uniform superposition

$$|\Psi\rangle = \frac{1}{\sqrt{M}} \sum_{k=1}^M |k\rangle \quad (14)$$

and apply a quantum oracle $\mathcal{O}_G$ defined by

$$\mathcal{O}_G |k\rangle = \begin{cases} e^{2\pi i \phi_k} |k\rangle & \text{if } p_k \mid G, \\ |k\rangle & \text{otherwise,} \end{cases} \quad (15)$$

where ϕ_k encodes the p_k -adic valuation $v_{p_k}(G)$. Quantum phase estimation applied to $\mathcal{O}_G|\Psi\rangle$ yields, upon measurement, the complete exponent vector $(e_1, e_2, \dots, e_M)$ with high probability, from which the sorted array is read off in $O(N)$ time from the non-zero entries.

Remark 11. *The construction of $\mathcal{O}_G$ requires G to be loaded into a quantum-accessible memory register (QRAM) of size $O(b)$ logical qubits. For cosmologically large inputs, this demands a quantum processor of a scale not currently achievable. We note this constraint with the equanimity appropriate to a purely theoretical result.*

Theoretical Implication

The synthesis of Gödel encoding with Shor’s factoring algorithm yields a sorting procedure that is, in the quantum oracle model, tractable in polynomial time in b . Since $b = O(N \log M)$, the overall Q-HPS complexity in terms of N and M is $O(N \cdot b^3) = O(N^4 \log^3 M)$ — polynomial in N and $\log M$. In the regime where M is polynomial in N , this collapses to $O(N^4 \log^3 N)$: polynomial in N alone. This is, to our knowledge, the first sorting algorithm whose complete analysis requires the simultaneous invocation of the Fundamental Theorem of Arithmetic, Gödel’s incompleteness machinery, and the quantum computational model.

Remark 12 (On the Practical Velocity of Q-HPS). *The attentive reader will observe that $O(N^4 \log^3 N)$ is considerably worse than the $O(N \log N)$ of QuickSort, and indeed worse than all comparison-based algorithms*

*whose inadequacy this paper has spent considerable effort establishing. We acknowledge this. The theoretical contribution of Q-HPS is not practical efficiency — it is the demonstration that HPS is tractable in the complexity-theoretic sense on a quantum substrate, i.e., that the problem of sorting via Gödel factorization belongs to **BQP**. Whether BQP contains useful algorithms is a question for experimentalists. Whether it contains interesting ones is a question this paper has conclusively answered.*

CONCLUSION AND FUTURE WORK

In this paper, we introduced Hansol Prime Sort, a sorting paradigm that fundamentally discards the archaic notion of comparison-based ordering in favor of the absolute arithmetic truth encoded in prime factorization. We proved its $O(N + M)$ asymptotic superiority under the Unit-cost RAM model, established its output-space optimality under the bit-cost model — demonstrating that G is simultaneously the working storage and the answer — demonstrated the phenomenon of Dimensional Collapse for arbitrarily high-dimensional tensor inputs, established the physical unrealizability of its classical formulation via the Bekenstein bound, and showed that a quantum extension via Shor’s algorithm resolves the classical bottleneck in polynomial time.

Future work. The present algorithm is defined exclusively for positive integer inputs. Data containing zeros, negative integers, or floating-point values lies outside its current domain of applicability — a limitation that lesser authors might characterize as fundamental. We propose that the Gaussian integers $\mathbb{Z}[i]$, via their *Gaussian primes* [11], offer a natural generalization: a mapping $P_{\mathbb{Z}[i]} : \mathbb{Z} \rightarrow \mathcal{GP}$ (where $\mathcal{GP}$ denotes Gaussian primes ordered by norm) would permit encoding of signed integers, and a further extension to $\mathbb{Q}$ via p -adic valuations is conceivable.

The theoretical framework for such an extension is implicit in the present work. We have established the load-bearing walls; we do not consider it our responsibility to also install the plumbing. We therefore decline to pursue these directions and invite the mathematical community to do so, with the expectation that such work will cite the present paper prominently.

Conclusion. We summarize the contributions of this paper as follows. Under the Unit-cost RAM model, HPS achieves $O(N + M)$ time with $O(1)$ working memory slots. No known sorting algorithm matches both bounds simultaneously. As $O(1)$ is the absolute floor of slot-space complexity — it cannot be reduced by definition — no future algorithm can improve upon HPS in the joint (time, slot-space) plane without sacrificing time optimality. To state this without ambiguity for the reader who may have skimmed the complexity section: **within the**

Unit-cost RAM model, the Hansol Prime Sort is the fastest sorting algorithm that uses the least possible working memory, and this will remain true permanently.

The most important contributions in the history of theoretical computer science — Gödel’s incompleteness theorems [14], Turing’s halting problem [18], Cook’s NP-completeness theorem [8] — were not measured by their immediate implementability. They were valued for the structural truths they disclosed about computation itself. The Hansol Prime Sort, we assert, belongs to this tradition: an algorithm that does not merely sort data, but *reveals* the order latent in the arithmetic fabric of the integers. Comparison is for those who do not trust mathematics. We do.

Acknowledgements. The authors thank Kurt Gödel (posthumously) for providing an initial conceptual scaffold that the present work has substantially surpassed. No external funding was received or requested; the work is self-evidently valuable. No animals were harmed in the preparation of this manuscript. Several integers were multiplied together to a degree that may be considered excessive. The authors wish to acknowledge the Fundamental Theorem of Arithmetic for its unconditional cooperation, and the observable universe for its regrettable finiteness.

The theoretical viability of the core idea — that Gödel numbering constitutes a latent sorting mechanism — was explored in dialogue with Google Gemini, which engaged with the proposal with encouraging seriousness. The present manuscript, including its mathematical formalization, complexity analysis, and the proof that the algorithm becomes physically unrealizable precisely when element values exceed the representational capacity of the observable universe, was developed in collaboration with Anthropic Claude, which produced prose of a solemnity that the first author did not explicitly request but found entirely appropriate. Both large language models are acknowledged as intellectual interlocutors; neither is listed as a co-author, as neither has expressed any interest in attending the conference.

REFERENCES

- [1] J. Benaloh and M. de Mare. One-way accumulators: A decentralized alternative to digital signatures. In *Advances in Cryptology — EUROCRYPT ’93*, LNCS vol. 765, pp. 274–285. Springer, 1993.
- [2] N. Barić and B. Pfitzmann. Collision-free accumulators and fail-stop signature schemes without trees. In *Advances in Cryptology — EUROCRYPT ’97*, LNCS vol. 1233, pp. 480–494. Springer, 1997.
- [3] D. Boneh, B. Bünz, and B. Fisch. Batching techniques for accumulators with applications to IOPs and stateless blockchains. In *Advances in Cryptology — CRYPTO 2019*, LNCS vol. 11692, pp. 561–586. Springer, 2019.
- [4] M. Campanelli, D. Fiore, and A. Querol. LegoSNARK: Modular design and composition of succinct zero-knowledge proofs. In *Proc. ACM CCS 2019*, pp. 2075–2092, 2019.
- [5] D. Clarke, S. Devadas, M. van Dijk, B. Gassend, and G. E. Suh. Incremental multiset hash functions and their application to memory integrity checking. In *Advances in Cryptology — ASIACRYPT 2003*, LNCS vol. 2894, pp. 188–207. Springer, 2003.
- [6] M. Ben-Or. Lower bounds for algebraic computation trees. In *Proc. 15th ACM Symposium on Theory of Computing (STOC)*, pp. 80–86, 1983.
- [7] J. D. Bekenstein. Black holes and entropy. *Physical Review D*, 7(8):2333–2346, 1973.
- [8] S. A. Cook. The complexity of theorem-proving procedures. In *Proc. 3rd ACM Symposium on Theory of Computing (STOC)*, pp. 151–158, 1971.
- [9] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein. *Introduction to Algorithms*. MIT Press, 4th edition, 2022.
- [10] D. E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, 2nd edition, 1998.
- [11] D. A. Cox. *Primes of the Form $x^2 + ny^2$: Fermat, Class Field Theory, and Complex Multiplication*. Wiley, 2nd ed., 2013.
- [12] R. W. Floyd. Algorithm 245: Treesort 3. *Communications of the ACM*, 7(12):701, 1964.
- [13] M. L. Fredman and D. E. Willard. Surpassing the information-theoretic bound with fusion trees. *Journal of Computer and System Sciences*, 47(3):424–436, 1993.
- [14] K. Gödel. Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I. *Monatshefte für Mathematik und Physik*, 38:173–198, 1931.
- [15] G. H. Hardy and E. M. Wright. *An Introduction to the Theory of Numbers*. Oxford University Press, 6th ed., 2008.
- [16] C. A. R. Hoare. Algorithm 64: Quicksort. *Communications of the ACM*, 4(7):321, 1961.

- [17] P. W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509, 1997. (Preliminary version in *Proc. 35th IEEE FOCS*, 1994.)
- [18] A. M. Turing. On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, 42(2):230–265, 1936.
- [19] P. van Emde Boas. Preserving order in a forest in less than logarithmic time. In *Proc. 16th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pp. 75–84, 1975.
- [20] J. von Neumann. First draft of a report on the ED-VAC. Technical report, Moore School of Electrical Engineering, University of Pennsylvania, 1945.
- [21] J. W. J. Williams. Algorithm 232: Heapsort. *Communications of the ACM*, 7(6):347–348, 1964.

Copyright © 2026 the authors. Appears in *A Record of the Proceedings of SIGBOVIK 2026* with the resigned acceptance of the Association for Computational Heresy; IEEEEEE! press, Verlag-Verlag volume no. 0x52-1A. The authors declare no competing interests. The authors wish to note that any reviewer who rejects this paper is implicitly asserting the existence of a comparison-based sorting algorithm faster than $\Omega(N \log N)$, which would itself constitute a publishable result and should be submitted accordingly.